ORDENAÇÃO

TÓPICO 8

PARTE 1

@projetotouuberlandia

Universidade Federal de Uberlândia

PROEXC

direc diretoria de extensão

Faculdade de Computação

TOU <Treinamento Olímpico de Uberlândia>

UBERHUB <CODE CLUB/>

O QUE VAMOS APRENDER?

- Aprenderemos sobre uma das operações mais importantes da computação: ordenação.
- Imagine um vetor {3,2,6,5} e você deseja ordená-lo em ordem crescente ou decrescente. Existem várias maneiras em que podemos ensinar um computador a ordenar esse vetor.
- Aqui ensinaremos como resolver esse problema de ordenação de duas formas, e da forma mais otimizada possível, em O(N * log N).

PORQUE APRENDER?

- Aprender sobre ordenação é importante não apenas por sua utilidade direta em muitos contextos computacionais, mas também por seu papel fundamental na compreensão de algoritmos e na resolução de problemas de forma eficiente.

ESTRUTURA BÁSICA

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cin >> n;
    // crio o vetor de n elementos
    int v[n];

    for(int i=0;i<n;i++){
        // leio cada variavel no vetor
        cin >> v[i];
    }

    // ordeno as n posições do vetor
    sort(v,v+n);

    for(int i=0;i<n;i++){
        // imprimo cada elemento
        cout << v[i] << " ";
    }
    // e a quebra de linha, no fim
    cout << "\n";
}
```

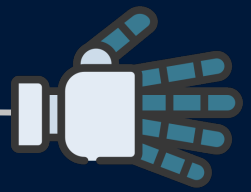
- Primeira Forma: usamos a função sort própria do C++, derivada da biblioteca bits.
- O sort( ) é chamado da seguinte forma: sort(inicio,fim);, onde inicio é um ponteiro para a primeira posição que queremos ordenar e fim um ponteiro para a primeira posição que não queremos ordenar.
- Portanto, o vetor será ordenado de forma crescente desde a posição 0 até n-1.

ESTRUTURA BÁSICA

```
// v = vetor original
int v[1000],aux[1000];
void mergesort(int l, int r){ // inicio e fim
    if(l==r) return; // chegou ao fim
    int m = (l+r)/2; // meio do segmento
    mergesort(l,m); // do inicio ao meio
    mergesort(m+1,r); // do meio+1 ao fim
    int pos1 = l;
    int pos2 = m+1;
    for(int i=l;i<=r;i++){
        if(pos1 == m+1){
            aux[i] = v[pos2]; pos2++;
        }
        else if(pos2 == r+1){
            aux[i] = v[pos1]; pos1++;
        }
        else if(v[pos1] <= v[pos2]){
            aux[i] = v[pos1]; pos1++;
        }
        else{
            // qtd += (meio+1) - pos1;
            aux[i] = v[pos2]; pos2++;
        }
    }
    for(int i=l;i<=r;i++) v[i] = aux[i];
}
```

- Segunda Forma: de forma manual, podemos ordenar o vetor chamando a função Merge Sort.
- O Merge Sort divide o vetor em duas partes sucessivamente, até cada segmento possuir apenas um único elemento (isso é feito de forma recursiva). Depois de separar todos, o Merge Sort une parte por parte do vetor de forma ordenada, até o vetor estar completamente ordenado.
- Por que usar? Ele faz o mesmo que o sort(), porém pode ser usado também para fazer uma contagem de inversões.

MÃO NA MASSA



1 Faça um algoritmo em C++ que ordene um vetor de forma crescente e depois mostre o vetor na tela.

2 Faça um algoritmo em C++ que ordene um vetor de forma decrescente e depois mostre o vetor na tela.

3 Faça um algoritmo em C++ que ordene um vetor e depois mostre o menor número.

MÃO NA MASSA

1 - NepsAcademy Ordenação Simples
https://neps.academy/br/course/programacao-basica-(codcad)/lesson/ordenacao-simples

2 - NepsAcademy Matryoshka
https://neps.academy/br/course/programacao-basica-(codcad)/lesson/matryoshka

3 - NepsAcademy Ordenação de Três Números
https://neps.academy/br/exercise/151

OBRIGADO

@projetotouuberlandia

Universidade Federal de Uberlândia

PROEXC

direc diretoria de extensão

Faculdade de Computação

<Treinamento Olímpico de Uberlândia>

UBERHUB <CODE CLUB/>

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**

Universidade Federal de Uberlândia

PROEXC Pró-reitoria de Extensão e Cultura

direc diretoria de extensão

Faculdade de Computação

TOU <Treinamento Olímpico de Uberlândia>

UBERHUB <CODE CLUB/>